

Retrieval-Augmented Generation (RAG) Tutorial

A comprehensive, hands-on tutorial that takes you from the basics through to building production-ready RAG systems.

■ What You'll Learn

By the end of this tutorial, you'll understand: - What RAG is and how it contributes to AI applications - How vector embeddings capture semantic meaning - How to build and query vector databases - How to integrate retrieval with language models - Advanced techniques for production RAG systems

■ Who This Is For

- **Developers:** Learn to build RAG applications
- **ML Engineers:** Understand the architecture and implementation details
- **Product Managers:** Gain technical insight into RAG capabilities

■ Learning Path

Part 1: Foundations

1. **Introduction to RAG**

2. The problem RAG solves

3. How RAG works at a high level

Real-world applications

Understanding Embeddings

6. What are vector embeddings?

7. Semantic similarity

Embedding models

Vector Databases & Retrieval

10. Storing and indexing vectors

11. Similarity search algorithms

Retrieval strategies

Language Models & Generation

14. How LLMs work

15. Prompting techniques
16. Integrating retrieved context

Part 2: Building RAG Systems

1. Building Your First RAG System

2. Step-by-step implementation
3. Working code example

Testing and evaluation

Advanced RAG Techniques

6. Document chunking strategies
7. Re-ranking retrieved results
8. Hybrid search
9. Query optimization
10. Handling edge cases

Part 3: Practical Examples

- [Example 1: Simple Question Answering](#)
- [Example 2: Document Chat System](#)
- [Example 3: Advanced RAG Pipeline](#)

■■ Setup

Prerequisites

- Python 3.13 (or 3.10+)
- Basic Python knowledge
- Understanding of APIs (helpful but not required)

Installation

1. Clone this repository:

```
git clone <repository-url>  
cd myproject
```

1. Create and activate virtual environment:

```
python -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate
```

1. Install dependencies:

```
pip install -r requirements.txt
```

1. Set up API keys (you'll need at least one):

```
# Create a .env file
cp .env.example .env

# Add your API keys:
# OPENAI_API_KEY=your-key-here
# Or for free alternatives:
# HUGGINGFACE_API_KEY=your-key-here
```

■ How to Use This Tutorial

1. **Start with Lesson 1** and work through sequentially
2. **Read the theory** in each lesson markdown file
3. **Run the code examples** to see concepts in action
4. **Experiment** with the examples and modify them
5. **Build your own** RAG system for your use case

■ Learning Tips

- **Take your time:** RAG combines multiple complex concepts
- **Run every code example:** Hands-on practice is crucial
- **Experiment:** Modify parameters and see what happens
- **Build projects:** Apply what you learn to real problems
- **Ask questions:** Open issues if something isn't clear

■ What is RAG?

Retrieval-Augmented Generation is a technique that enhances large language models by giving them access to external knowledge. Instead of relying solely on their training data, RAG systems:

1. **Retrieve** relevant information from a knowledge base
2. **Augment** the LLM's prompt with this information
3. **Generate** accurate, grounded responses

This solves key problems like: - ■ Hallucinations (making up facts) - ■ Outdated information - ■ Lack of domain-specific knowledge - ■ Inability to cite sources

Ready to start? Head to [Lesson 1: Introduction to RAG](#)